

Projet Architecture des Ordinateurs

Geometry Dash

L3 Informatique – INU Champollion – 2026

Angel BESANCENEZ – Chris KALOUCHE

1. Présentation du projet

Ce projet associe un jeu Geometry Dash développé en Python avec une matrice LED RGB 64×64 pilotée par un Arduino. Le jeu tourne sur PC et envoie en temps réel des informations de progression à l'Arduino via le port série USB. La matrice affiche alors le numéro du niveau en cours, une barre de progression et le compteur de morts.

2. Architecture générale

Le projet est découpé en deux parties distinctes qui communiquent par liaison série :

- Partie 1 – Jeu Python (PC) : logique de jeu complète, gestion des collisions, rendu graphique Pygame, envoi des données série.
- Partie 2 – Affichage Arduino (matrice LED) : réception des paquets série, parsing, et affichage sur la matrice DFRobot RGBMatrix 64×64.

Le protocole de communication est minimaliste : des paquets texte courts terminés par un retour à la ligne (`\n`), ce qui évite tout overhead et garantit une réception fiable.

Paquet	Description
I : 2	Numéro du niveau (envoyé une seule fois au démarrage)
P : 42	Progression en pourcentage (envoyé toutes les 0,5 secondes)
D : 7	Nombre de morts (envoyé uniquement à chaque mort)

3. Description des fichiers

Fichier	Rôle
<code>game.py</code>	Boucle principale du jeu Geometry Dash. Gère le rendu Pygame, la physique, les collisions, et envoie les données à l'Arduino.
<code>world_generation.py</code>	Lecture des niveaux CSV, construction de la carte, détection des collisions (blocs, spikes, orbes, pads), dessin du joueur.
<code>arduino_sender.py</code>	Module de communication série. Détecte automatiquement le port Arduino et expose les fonctions d'envoi (level, progression, morts).
<code>arduino_matrix.ino</code>	Programme Arduino. Reçoit les paquets série, les parse, et met à jour l'affichage de la matrice LED uniquement si les valeurs changent.
<code>maps/level_X.csv</code>	Fichiers de niveaux encodés en CSV. Chaque cellule représente un élément du jeu (0=bloc, S=spike, J=orbe, P=pad, B=joueur).

3.1 game.py

C'est le point d'entrée du programme. Il initialise Pygame, la connexion Arduino, charge les assets graphiques et lance la boucle de jeu. La connexion série et l'envoi du numéro de niveau sont effectués une seule fois au niveau du module (hors de toute fonction) pour éviter les renvois inutiles lors des relances.

À chaque mort, la variable `deaths` est incrémentée et le nouveau total est immédiatement envoyé à l'Arduino. La progression (`camera_x / largeur totale`) est calculée et envoyée toutes les 0,5 secondes.

3.2 arduino_sender.py

Module autonome sans classe. Il expose trois fonctions d'envoi simples.

3.3 arduino_matrix.ino

Le programme Arduino lit le port série caractère par caractère et reconstruit les paquets jusqu'au retour à la ligne. Chaque zone de l'écran (level, barre de progression, morts, avatar) ne se redessine que si la valeur reçue diffère de la dernière valeur affichée, grâce aux variables `lastPct`, `lastMorts` et `lastLevel`.

L'avatar du personnage est stocké dans un tableau `uint8_t[16][16][3]` et affiché en deux boucles `for`. Il est dessiné une seule fois au démarrage lors de la réception du paquet de niveau.

4. Comment lancer le projet

4.1 Prérequis

- Python 3.10+ avec les bibliothèques : `pygame`, `pyserial`
- Arduino IDE avec la bibliothèque `DFRobot_RGBMatrix` installée
- Arduino branché en USB avant de lancer le jeu

Installation des dépendances Python :

```
pip install pygame pyserial
```

4.2 Étapes de lancement

- **Étape 1 – Uploader arduino_matrix.ino sur l'Arduino via l'IDE Arduino.**
- **Étape 2 – Vérifier le port série dans arduino_sender.py (ligne arduino_init).**
- **Étape 3 – Lancer le jeu Python :**

```
python game.py
```

- **La matrice affiche automatiquement le niveau, la barre de progression et le compteur de morts dès le démarrage du jeu.**

5. Disposition de l'affichage

La matrice 64×64 est divisée en quatre zones fixes qui ne se superposent pas :

Zone	Pixels (Y)	Contenu
Niveau	0 – 8	"Level : X" en vert
Progression	10 – 18	Barre verte qui avance de 0 % à 100 %
Avatar	20 – 36	Sprite 16×16 px du personnage (pixel art généré depuis l'image du jeu)
Morts	56 – 64	"Morts : X" en rouge — persiste entre les tentatives

6. Points à améliorer

Le projet fonctionne mais plusieurs améliorations sont envisageables :

- Barre de progression : le fillRect qui efface la zone avant de redessiner peut encore provoquer un léger clignotement si les paquets arrivent trop fréquemment. Une solution serait de calculer la progression coté Arduino.
- Fréquence d'envoi adaptative : actuellement fixée à 0,5 seconde. On pourrait envoyer la progression uniquement lorsque le pourcentage entier change, réduisant encore le trafic série.
- Écran de fin de niveau : lorsque le joueur termine un niveau, la matrice ne le signale pas encore. On pourrait envoyer un paquet spécial (ex. WIN:1) pour déclencher une animation sur la matrice.
- Multiples niveaux dynamiques : la largeur des niveaux est codée en dur dans un dictionnaire. Si un nouveau niveau CSV est ajouté, il faut penser à mettre à jour LEVEL_WIDTHS dans game.py.

7. Conclusion

Ce projet illustre concrètement la communication entre un processeur hôte (PC) et un microcontrôleur (Arduino) via liaison série. La séparation des responsabilités — le PC gère la logique du jeu, l'Arduino gère exclusivement l'affichage — permet à chacun de fonctionner à son propre rythme sans blocage mutuel.

La principale difficulté rencontrée a été la gestion du clignotement de la matrice LED : chaque redessinage provoque un flash visible. La solution adoptée (redessiner uniquement les zones dont la valeur a changé) réduit considérablement ce problème.

L'ensemble représente une application complète du cours d'architecture des ordinateurs : protocole de communication, contraintes temps-réel, gestion des entrées/sorties et programmation bas niveau sur microcontrôleur.

Note : pour certaines questions techniques complexes (débogage de la communication série, optimisation de l'affichage matrice, parsing de paquets Arduino), nous avons utilisé l'IA Claude d'Anthropic comme outil d'aide et de test.